#A Create a test user.
#B Login to get a token with the HMAC tag.

If you try and use the token without the authentication tag, then it is rejected with a 401 response. The same happens if you try to alter any part of the token ID or the tag itself. Only the full token, with the tag, is accepted by the API.

5.3.3 Protecting sensitive attributes

Suppose that your tokens include sensitive information about users in token attributes, such as their location when they logged in. You might want to use these attributes to make access control decisions, such as disallowing access to confidential documents if the token is suddenly used from a very different location. If an attacker gains read access to the database, they would learn the location of every user currently using the system, which would violate their expectation of privacy.

Encrypting database attributes

One way to protect sensitive attributes in the database is by encrypting them. While many databases come with built-in support for encryption, and some commercial products can add this, these solutions typically only protect against attackers that gain access to the raw database file storage. Data returned from queries is transparently decrypted by the database server, so this type of encryption does not protect against SQL injection or other attacks that target the database API. You can solve this by encrypting database records in your API before sending data to the database, and then decrypting the responses read from the database. Database encryption is a complex topic, especially if encrypted attributes need to be searchable, and could fill a book by itself. The open-source *CipherSweet* library (https://ciphersweet.paragonie.com) provides the nearest thing to a complete solution that I am aware of but lacks a Java version at present.